# Prayog — AI Engineering Log

CS455 Deliverable 3 · Group 3 · consolidated 6 November 2026

## 1. How this log is organised

Prayog was built with AI coding assistants throughout. This consolidated log covers the whole project; the detailed per-phase logs remain the primary records and are referenced, not repeated:

| Phase | Dates | Record | Owner |
|---|---|---|---|
| Requirements, architecture, verification (D1) | Sep 2026 | SADD v2.0 ADR sections, verification package (`final deliverables/`), section 2 below | team |
| ML Flow lifecycle and Steward handoff (D2) | 12–19 Oct | `AI_ENGINEERING_LOG.md` in the repository (5 evaluated recommendations, PRs #2–#14 on the working repo) | Nideesh |
| Steward S1–S4 (D2) | 5–15 Oct | `Steward_AI_Engineering_Log.md` (five phases, 14 AI errors caught) | Areen |
| Integration, web app, code review (D2→D3) | 29 Oct – 5 Nov | section 3 below | Areen |
| GPU scheduler and node lifecycle (D2) | 12–15 Oct | prompt-by-prompt log kept by Adithya; repository history of PRs #4–#9 | Adithya |
| Testing, security, performance, reliability (D3) | 6 Nov | section 4 below | Areen, with Claude Code |
| GPU subsystem: tests, experiments and defect fixes (D3) | 6 Nov | section 5 below | Adithya, with an AI coding agent |

**Tools.** Claude Code (desktop app, Claude Opus 5.5) for most code, tests and documents; 42 commits on `main` carry a Claude co-author trailer (38 Opus 5.5, 4 Sonnet 5.5). Teammates' commits without a trailer may also be AI-assisted; their own logs are the record for that work. **Rule kept throughout:** the human sets the goal and takes every decision that is theirs (scope, design choices, what to push, what to submit); the AI proposes, implements and verifies; nothing counts as done until it passes tests on real PostgreSQL; nothing is pushed without explicit confirmation.

## 2. Requirements and architecture (D1)

| Activity | AI contribution | Human decision / check | Outcome |
|---|---|---|---|
| Cross-document verification of three SRSs and SADD v1.0 | found 18 architecture defects (AD-01…18) and the cross-document conflicts (consistency matrix) | team accepted the register; decided the Integrated Requirements Specification is authoritative | `final deliverables/04_Consolidated_Defect_Register.md` |
| SADD v2.0 | drafted Parts I–IV, 41 mermaid diagrams, 29 ADRs, Appendix A tracing all 400 requirement IDs | reviewed; `tools/check_trace.py` verifies the trace mechanically | v2.0 written; note: the D1 submission carried v1.1 |
| Backlog and sprint plan | first draft put 175 points in Sprint 1 against 130 capacity (69 on one person) | rejected and rebalanced to 134 points before publication | `PRAYOG_Backlog_and_Sprint_Plan.md` section 4 |

## 3. Integration, web app and full code review (29 Oct – 5 Nov)

| # | Prompt (condensed) | AI output | Evaluation | Decision |
|---|---|---|---|---|
| 1 | "Re-check the Steward against the final product" | fresh clone of `main`; ran every suite; traced requirements to code | found that CLI rebuilds create no `RebuildAttempt`, so failed CLI rebuilds never reach S3/S4 (FR-STW-001/006, FR-INT-01) | fixed in `8a38f79` with regression test `637bf75` (defect D3-01) |
| 2 | "Build a web UI for the Steward for every role" | no-build JS single-page app; new `GET /steward/diagnoses` | 17 integration tests; live run in the browser pane | human chose all four role areas and no-build JS over React |
| 3 | "Build the Prayog web app for every role" | `backend/web` served at `/app`; `GET /jobs`, `GET /mlflow/handovers`; 10 tests | live run with seeded data | human chose one shell over two linked apps |
| 4 | "Do a full code review" | 27 findings | each finding reproduced before fixing; 21 fixed with tests (`test_review_fixes.py`, 14 tests), 6 left open and listed | fixes in `717f1fc` (defects D3-02, D3-09, D3-10, D3-11) |
| 5 | "Fix CI" on PR #26 | diagnosed `ubuntu-latest` = 26.04 without Python 3.9, and the e2e user lacking the new access rule | CI green on `cc209d9` | merged as PR #26 (D3-12) |

## 4. Deliverable 3 session (6 Nov)

**Opening prompt (verbatim, from the user):** the course's Deliverable 3 instructions pasted as-is, then the answers to three questions the AI asked before starting: scope "*Whole team package*", format "*PDFs + one zip*", Jira "*I'll export now*".

| # | What the AI did | How it was checked | Result |
|---|---|---|---|
| 1 | Ran the backend, CLI and Steward suites with coverage on a fresh checkout of `main` | junit XML + coverage XML kept as evidence | Test and Coverage reports |
| 2 | Wrote six concurrency tests (barrier-released threads, one session each) | each run three times; C6 also run against the unfixed code to show it fails | found **D3-03**: one inconsistent unit row makes every scheduling pass crash |
| 3 | Wrote live failure drills (kill agent / API / DB / tick) | timeline JSON + database inspection afterwards | found **D3-08** (outage looks like node failure) and **D3-06** (500 on DB outage) |
| 4 | Wrote a closed-loop load generator; measured baseline and optimised builds side by side | same database, same machine, sequential runs | found **D3-07** (pool exhaustion) and quantified **D3-04** (N+1) |
| 5 | Implemented batched `GET /jobs`, pool sizing, 503 handler, claim hardening, workspace permissions | new tests fail on the old code and pass on the new; full suites re-run | five fixes on branch `d3-testing` |
| 6 | Ran bandit and pip-audit, triaged every Medium/High finding by reading the code | — | **D3-05** real; 5 false positives explained |
| 7 | Wrote 22 adversarial probes and ran them against the live API | each probe states the expected safe behaviour before it runs | Security report section 7 |
| 8 | Drafted all D3 documents from the measured data | every number traced to a results file in `evidence/` | this package |

### 4.1 Where the AI was wrong this session, and how it was caught

| # | AI error | Caught by | Correction |
|---|---|---|---|
| 1 | Concurrency tests read `user.id` from an ORM object inside worker threads, which made SQLAlchemy refresh it through the shared test session: "*This session is provisioning a new connection; concurrent operations are not permitted*" | the first test run | read every ID once before starting threads; comment in the test explains why |
| 2 | C1/C2 assumed a node existed; on a fresh database every submission was refused ("largest node you can use has 0") | running on a clean database instead of the shared one | each test creates its own node |
| 3 | C5 asserted that a cancelled job's node had no busy units, but other tests' queued jobs legitimately took them | inspecting the database rows behind the failure | assert only that no unit points at the cancelled job's allocations |
| 4 | C6 first used a shared node, so leftover queued jobs from other tests took it and the test failed for an unrelated reason | inspecting who held the "good" node | nodes reserved to the test's own lab |
| 5 | The batched-list fixture put a maintenance window inside a queued job's walltime, which legitimately suppresses `p0_free_at` | reading `p0_guaranteed_free_at` | window moved after the queued job's walltime; a separate fully-busy node gives the non-null case |
| 6 | The first load generator crashed on the first transport error instead of counting it | the baseline run aborting with `RemoteProtocolError` | errors are now recorded per status/exception; results are streamed to a `.partial` file |
| 7 | C3's final assertion assumed its jobs could only land on its own node; in the full suite on a shared database, passes placed them on other tests' shared nodes | the final full-suite run (13 failures instead of the expected 12) | the assertion counts live allocations on any node; re-run on both databases |
| 8 | The drill re-run reused the simulated agent's state directory from the first run, whose credential belonged to a dropped database; together with a concurrency run started at the same time, the job never left QUEUED and the script crashed, leaving its processes running | the drill timeline and the orphaned tick still logging | state directory cleared per run; the script now always kills its processes (`try/finally`); nothing else runs during drills |
| 9 | A first reading of the 4 `NODE_DOWN` alerts after 2 agent kills could have been dismissed as noise | cross-checking every alert against the drill timeline and the allocation history | identified as defect D3-08, not a test artefact |

### 4.2 AI suggestions rejected or deferred

| Suggestion | Why not (now) |
|---|---|
| Fix D3-08 (reaper grace after control-plane outage) in this session | changes the node-failure semantics owned by the GPU subsystem (FR-GPU-077); needs the owner's design decision. Left to the owner, and fixed in section 5. |
| Add a new `AlertKind` for refused claims | needs a migration and an admin-UI change; a logged ERROR is enough for now |
| Replace `xml.etree` with `defusedxml` | low risk; recorded as S-03 |
| Report coverage only from the Linux CI run | CI artifacts are not readable without signing in; local coverage reported instead, with the Windows-only failures named |

## 5. GPU subsystem session (6 Nov)

This session started from `main` while Areen's session ran in parallel. When his package arrived, the work was moved onto his branch and merged with it.

## 5.1 Prompts, outputs and decisions

| # | Prompt (verbatim) | What the AI did | How it was evaluated | Decision |
|---|---|---|---|---|
| 1 | "Check the current status of project now. We need to get all of deliverable 3 done" | Read the course's D3 list from the PDF and compared it with the repository, CI and the local tests | Every status claim tied to a command's output | Accepted. It found that 68 of the 333 local scheduler tests failed on current `main` |
| 2 | (same task) | Traced the 68 failures to one cause: submitting now needs an approved access request. Ported the test factory | 332 of 333 then passed; the last needed the seed script's new signature | Accepted. It also caught that the tests would have dropped CI's shared database, so they now use their own |
| 3 | (same task) | Wrote hostile probes through the real API, then failing tests for each defect, then the fixes | Failing output saved before any fix (`evidence/adversarial_before_fix.txt`); the same tests pass after | Accepted: D3-16 to D3-20, including an authorization hole (D3-18) |
| 4 | (same task) | Ran a pass benchmark and found four overlapping passes took 51 s at 400 jobs. Proposed remembering a lost race within the pass | Regression test first failed with 90 claims instead of 3; benchmark repeated | Accepted (D3-14) |
| 5 | (same task) | Load test with agents showed collapse at 50 users. Proposed bounding passes per process | Three-way comparison of `main`, `d3-testing` and the final build, each on an empty database | Accepted (D3-15) |
| 6 | "A zip for the deliverable has already been done by areen, check that once, and ensure to improve on it based on our efforts and give a clear report finally of whatever you have done" | Read all 14 documents and his branch, moved the work onto `d3-testing`, and updated the package rather than replacing it | Merge conflict in `main.py` and `jobs.py` resolved by hand; all suites re-run on the merged code | Accepted |
| 7 | (same task) | Fixed the two defects his package left for the GPU owner: D3-08 and D3-13 | D3-08 reproduced at production timings before the fix, and shown gone after | Accepted. D3-13 is a design choice recorded in the Security report |

## 5.2 Where the AI was wrong, and how it was caught

| # | AI error | Caught by | Correction |
|---|---|---|---|
| 1 | The first load baseline ran while test suites were also running on the same machine, which contaminates the measurement | Noticed on review of its own order of work | Discarded that run. Every benchmark then ran alone, on an empty database |
| 2 | The first failure scenarios **did not reproduce D3-08**: a simulated agent heartbeating every 2 s reached the server before the next tick, and the API crash lasted only 7 s. Read at face value, this said the defect did not exist | Comparing the result with Areen's drill, which did see it | Added a 45 s API outage and the production 10 s heartbeat. The defect then reproduced every time |
| 3 | **The first three-way benchmark was invalid.** The image builds for `main` and `d3-testing` failed, and the script silently ran the final image in their place, labelled as baselines. It would have reported no difference between the builds | The numbers looked identical to the final build, and one log line said `No such image` | Results deleted. The script now stops if a build fails, and checks that each image really contains the code it should |

| # | AI error | Caught by | Correction |
|---|---|---|---|
| 4 | A regression test for out-of-range agent reports "failed" against unfixed code for the wrong reason: it used an allocation that did not exist, so it saw 404, not the real 500 | Reading why it failed, not just that it failed | Rewritten to use a real running allocation; it then showed the real 500 |
| 5 | The first version of the fix for a node leaving AVAILABLE referred to a node state that does not exist | Reading the enum before applying it | Marks the node's units as not free in this pass's view |
| 6 | A write of this package's text used "section " and other forms the team's writing rule forbids | The rule was already on record | Replaced throughout the package |

### 5.3 AI suggestions rejected

| Suggestion | Why not |
|---|---|
| A global advisory lock so only one pass runs at a time | NFR-GPU-006 forbids a global lock that serialises unrelated units, and NFR-GPU-007 expects passes from several workers. One running plus one owed **per process** bounds the storm and keeps the row-lock design |
| Validate `project_ref` on `POST /jobs` against ledger projects, as part of the D3-18 fix | A wider behaviour change on the deadline day, and the Steward already scopes projects by owner, so it does not cross tenants. Recorded as a known limitation |
| Report only the improvements from our own fixes | Areen's pool sizing and batched job list also lift the same numbers. The three-way comparison credits each change |
| Claim the cloud deployment as done because the image and compose stack work | Running locally is not deploying to a cloud service. It stays recorded as not done until someone runs it |

### 5.4 Review responsibility

Every fix carries a test that failed before it and passes after, and the performance fixes carry before-and-after measurements. They reach `main` only through a pull request made from Adithya's own account and reviewed by a teammate.

## 6. Lessons

1. **AI-written code that "works" still hides defects that only adversarial conditions reveal.** None of D3-03… D3-08 was visible to the 455 passing functional tests; all six came from concurrency, load, fault-injection and static-analysis runs.
2. **The AI's own tests are as fallible as its code.** Six of the nine errors in section 4.1 were in test code and two in the test harnesses (load generator, drill script). Every new test was therefore also run against the unfixed code to prove it can fail.
3. **Asking before acting saves time.** The three scoping questions at the start fixed scope and format before any work; the Jira data the AI cannot reach was requested from the human at once.
4. **Effort moved from writing to verifying** (Costing section 3): the size-based estimate is ten times the effort spent, and the remaining hours went to review and testing.
5. **A test that passes can simply be too gentle.** The first failure scenarios looked like good news about D3-08. Only a comparison with independent evidence showed they were not stressing the system at production timings.
6. **Two independent passes over the same system found different things.** Areen's session found the N+1 list and pool exhaustion; this session found the pass storm and the snapshot hole. Each would have missed the other's.